Routing-based SSRF in an E-commerce webapp

Ref: CVE-2024-21510

Executive Summary

This report outlined the successful exploitation of a Server-Side Request Forgery (SSRF) vulnerability caused by insecure routing behavior based on the `Host` header. The vulnerability allowed the tester to route internal requests to services within the internal `192.168.0.0/24` subnet. This flaw was used to access an internal admin panel and perform unauthorized actions, including deleting a user. This vulnerability is considered **high severity** due to its ability to bypass internal network boundaries and compromise sensitive internal functionality.

Introduction

The objective of this assessment was to identify and exploit SSRF vulnerabilities resulting from unsafe server-side routing mechanisms. Specifically, this test targeted the web application’s behavior when processing user-controlled `Host` headers in HTTP requests.

Methodology

1. Burp Suite was used to intercept and modify HTTP requests and observe the application's behavior.

2. Network interaction capabilities were tested using Burp Collaborator.

3. A brute-force scanning approach was conducted to discover accessible internal IP addresses.

4. HTTP request manipulation was used to access protected internal admin functionality.

5. Manual request crafting was used to bypass CSRF protections and delete a user.

Vulnerability Findings

- **Type**: Server-Side Request Forgery (Routing-based via Host header)

- **Location:** HTTP `Host` header

- **Severity**: High

Description

The application trusted the `Host` header to make routing decisions internally. By modifying this header to reference internal IP addresses (within the `192.168.0.0/24` range), the tester was able to route the request to an internal admin interface. The interface was unprotected from unauthorized users once accessed internally, allowing full access to administrative functions.

Proof of Concept (PoC)

1. A `GET /` request was modified in Burp Suite Repeater, and the `Host` header was set to a Burp Collaborator payload. DNS and HTTP interactions confirmed the backend issued external requests.

2. A burp Intruder scan iterated through the range `192.168.0.0` to `192.168.0.255` in the Host header:

makefile

CopyEdit

`Host: 192.168.0.§0§`

3. A 302 redirect to `/admin` was observed for a specific IP (e.g., `192.168.0.2`), confirming the internal admin panel's location.

4. The `/admin` panel was accessed successfully by modifying the request.

5. The CSRF token and session cookie were extracted from the response.

6. A crafted POST request was sent to `/admin/delete?csrf=[token]&username=user` with the session cookie set.

7. The `user` was successfully deleted.

Impact Assessment

Exploitation of this SSRF flaw allowed external attackers to:

- Access and manipulate internal admin functionality.

- Delete or modify users and data.

- Potentially pivot deeper into the internal network.

These risks can lead to full account takeover, privilege escalation, and internal system compromise, depending on what services are exposed internally.

Recommendations

1. Do not trust the `Host` header for routing logic or access control.

2. Enforce strict internal-to-external request boundaries using firewall rules.

3. Use network segmentation to isolate administrative endpoints from user-accessible components

4. Apply authentication and CSRF protection on all internal admin interfaces, regardless of network origin.

5. Regularly test for SSRF vulnerabilities using both **static and dynamic analysis** tools.

Conclusion

This assessment highlighted a serious SSRF vulnerability stemming from insecure trust in user-supplied `Host` headers. The ability to redirect internal server behavior enabled unauthorized access to internal administrative controls. Mitigation should be prioritized to avoid future exploitation and maintain a strong security posture.